

Click-to-Source 3D

Stage 3 Implementation & Validation Report

Runtime Resolution Engine, Selection Highlighting, Generation Trace & Edit/Write-Back

1. Executive Summary

Stage 1 proved that a runtime object could carry provenance metadata through the React Three Fiber lifecycle. Stage 2 converted those findings into a frozen architectural contract: the [SourceRef](#) shape and the JSX-declarative tagging rule. Stage 3 implements the runtime system that consumes that contract — the actual MVP loop described across all three planning documents:

click an object → resolve its source → display file/function/line/args → edit a value → write it to disk → let Vite HMR update the rendered scene.

This report consolidates two independent audits performed after implementation: a process-level account of how each step was built, what broke, and how it was diagnosed and fixed; and a code-level inventory reading the actual current source files, line by line, to confirm what is really implemented versus what was only claimed. Where the two disagreed, the code-level evidence is treated as authoritative, and both positions are recorded below rather than silently reconciled.

Stage 3 is functionally complete against its ten-step definition. Two matters are documented as known, real limitations rather than closed questions: the panel's behavior when the underlying source changes by a path other than its own save action, and the narrow scope of what a hand-tagged [SourceRef](#) currently exposes. Neither blocks Stage 4.

2. Stage 3 Objective

The objective was to turn the Stage 2 contract into a working interaction loop:

```
Developer clicks a 3D object
|
Identify the clicked THREE.Object3D
|
Resolve its SourceRef
|
Store the current selection
|
Visually highlight the selected object
|
Display its source information
|
Edit a value in the panel
```

```
|  
Write the edit to the source file  
|  
Let Vite HMR reload the scene
```

Stage 3 deliberately excluded automatic source-code instrumentation, AST-based generator discovery, AI integration, and MCP — all explicitly deferred to later stages per the project's Non-Goals discipline. The [SourceRef](#) records used throughout Stage 3 remain manually supplied, consistent with the Stage 2 v0.1 convention.

3. Architecture

The implementation spans four packages with a strict one-directional dependency chain:

```
shared -> core -> overlay -> examples
```

[shared](#) contains only the [SourceRef](#) contract:

```
export type SourceRef = {  
  file: string;  
  function: string;  
  line: number;  
  args: Record<string, unknown>;  
  schemaVersion?: number;  
};
```

[core](#) is framework-agnostic. It contains the provenance resolver and the Babel/magic-string source editor. It has no React, Zustand, or rendering-specific code.

[overlay](#) contains the React-facing layer: the R3F click hook, the Zustand selection store, the OutlinePass-based highlight component, the GenerationTrace panel, and the client that talks to the file-I/O endpoints.

[examples](#) is the integration environment: a small demo scene, the Vite dev-server plugin providing the local filesystem boundary, and OrbitControls for camera testing.

This separation was maintained throughout — the resolver has no knowledge of highlighting or UI, and the highlight component has no knowledge of how a [SourceRef](#) was produced.

4. Step-by-Step Implementation

Step 1 — Core resolution and raycast integration

Built: A framework-agnostic resolver in [packages/core](#), using [THREE.Raycaster](#). The important behavior is the parent-chain walk: a raycast may hit a child mesh while provenance metadata lives on a parent group, so the resolver climbs [object.parent](#) looking for [userData.sourceRef](#).

Confirmed in source ([packages/core/src/resolver.ts:20](#)):

```
export function resolveSourceRef(object: THREE.Object3D): ResolutionResult | null {
  let current: THREE.Object3D | null = object;
  while (current) {
    if (current.userData && current.userData.sourceRef) {
      return { object: current, sourceRef: current.userData.sourceRef as SourceRef };
    }
    current = current.parent;
  }
  return null;
}
```

It checks the clicked object first, then walks upward, returning the nearest match or `null`.

Problem found: the first R3F integration ignored the event R3F already produced and ran a second, redundant raycast against the scene.

Diagnosis: R3F's `ThreeEvent` already contains the intersected object (`event.object`) along with intersection, camera, ray, and pointer data. A second raycast was unnecessary and wasteful.

Fix: the provenance walk was extracted into the standalone `resolveSourceRef(object)` function shown above, decoupled from raycasting itself. The R3F hook was changed to pass `event.object` directly into it, eliminating the second raycast entirely.

Verification: 6 core tests passing at this step; TypeScript and Vite builds green.

Step 2 — R3F click integration and Zustand selection state

Built: `useClickToSource()` — intentionally narrow, receiving an R3F pointer event, pulling `event.object`, and delegating to the core resolver. Confirmed in source

(`packages/overlay/src/hooks/useClickToSource.ts:7`):

```
export function useClickToSource() {
  return function resolveClick(event: ThreeEvent<PointerEvent | MouseEvent>): ResolutionResult | null {
    return resolveSourceRef(event.object);
  };
}
```

No raycasting logic lives in the hook.

A Zustand store was added holding `selectedObject`, `sourceRef`, and `panelOpen`, with `select()`, `clearSelection()`, and `setPanelOpen()` actions. Confirmed shape

(`packages/overlay/src/store/overlayStore.ts:6`) additionally includes `updateSourceRefArg()`, added later to support Step 8's edit flow.

Problem found: the store imported Three.js types as runtime imports rather than type-only imports, and tests depended on Zustand's module-level state persisting between test runs — meaning test order affected results.

Fix: imports converted to type-only; a `beforeEach` reset was added restoring `selectedObject`, `sourceRef`, and `panelOpen` to their initial values before every test, making tests independent of execution order.

Verification: 4 overlay tests passing (initial state, selection, clearing, panel visibility while retaining selection); core's 6 tests still passing.

Step 3 — OutlinePass highlighting

Built: `SelectionHighlight`, using Three.js's native `EffectComposer`, `RenderPass`, and `OutlinePass`. Confirmed in source (`packages/overlay/src/components/SelectionHighlight.tsx:13`):

```
const { composer, outlinePass } = useMemo(() => {
  const composer = new EffectComposer(gl);
  const renderPass = new RenderPass(scene, camera);
  composer.addPass(renderPass);
  const outlinePass = new OutlinePass(new THREE.Vector2(size.width, size.height), scene, camera);
  composer.addPass(outlinePass);
  return { composer, outlinePass };
}, [gl, scene, camera]);
```

The `size` dependency is deliberately excluded from this `useMemo`, so viewport resizing does not tear down and rebuild the composer — `resize` is instead handled separately via `composer.setSize()`. Selection changes only mutate `outlinePass.selectedObjects`, confirmed as a single-line assignment rather than any composer rebuild. Rendering runs through `useFrame()` at a positive priority, and cleanup disposes both the composer and outline pass on unmount.

Problem found: uncertainty over whether the positive `useFrame` priority would cause a second, competing render pass alongside R3F's own automatic render.

Diagnosis: the installed R3F source was inspected directly rather than assumed. Positive priority causes R3F to skip its own automatic render for that path, handing rendering control to the composer — confirmed as a single render path, not two.

A separate, non-blocking visual observation was logged: outline thickness appears to change with camera zoom distance (thinner far away, thicker close up) — a screen-space characteristic of `OutlinePass` itself, not a defect, and intentionally left unmodified.

Fix: `OrbitControls` were added, scoped strictly to `packages/examples`, using Three.js's own built-in implementation rather than adding `@react-three/drei` as a new dependency, so camera behavior could be tested against the highlight system.

Verification (manual): selection transferred correctly between pink cube, cyan sphere, and untagged orange cube; background clicks cleared selection; rapid switching produced no stale outlines; the outline remained correctly attached through camera orbit, rotation, and zoom, including through occluding geometry (intentional, accepted `OutlinePass` behavior).

Step 4 — GenerationTrace display (read-only)

Built: a presentation-only component reading the Zustand store and displaying `file`, `function`, `line`, and `args`. It performs no raycasting, resolution, parsing, or file access. Confirmed in source (`packages/overlay/src/components/GenerationTrace.tsx:46`), it returns `null` when `panelOpen` is false or `sourceRef` is absent.

Problem found: the initial demo's hand-authored `SourceRef` line numbers (33, 39) did not correspond to any real editable location in `main.tsx`. This was correctly diagnosed as a fixture-data problem, not a resolver failure — the resolver was faithfully returning exactly the (wrong) metadata it had been given.

Fix: deferred until Step 7 made the actual required source locations concrete (see below).

Verification (manual): trace correctly displayed and updated across pink/cyan selection, cleared on untagged/background clicks, survived camera movement and rapid switching.

Step 5 — Vite development file-I/O plugin

Built: a development-only Vite plugin exposing two endpoints. Confirmed in source ([packages/examples/vite-file-io.ts:11](#)):

```
const READ_FILE_PATH = "/__cts/read-file";
const WRITE_FILE_PATH = "/__cts/write-file";
```

Guarded to development only via `apply: "serve"` ([vite-file-io.ts:280](#)). The implementation includes JSON validation, project-root path containment, traversal/absolute-path rejection, and symlink safety checks — treated as required implementation from the start rather than an afterthought.

Verification: endpoint tests passing; production bundle confirmed to exclude the dev-only routes; temporary test fixtures cleaned up after use.

Step 6 — Babel + magic-string source editing

Built: `editSource()` in [packages/core](#), parsing TSX/JSX with Babel and patching only the exact character range of a targeted literal using `magic-string`. Confirmed in source ([packages/core/src/sourceEditor.ts:298](#) for parsing, [:338](#) for the patch):

```
const range = getNodeRange(candidate.valueNode);
const replacement = serializeReplacement(candidate, request.newValue);
const magicString = new MagicString(source);
magicString.overwrite(range.start, range.end, replacement);
```

Supported value types: strings, finite numbers, booleans, and `null`. Unsupported types (objects, arrays, functions, non-finite numbers) produce a typed `UNSUPPORTED_VALUE` error rather than attempting a transformation. Candidate discovery covers both JSX attributes (string and expression forms) and object properties; when multiple literals match the same argument name, the target is disambiguated by line number, and a true ambiguity produces `AMBIGUOUS_LOCATION` rather than silently picking one. This is by design a narrow, single-literal editor — not a general AST transformation engine.

Problem found: when Step 7's edit flow was first attempted against the existing demo metadata ([main.tsx](#), lines 33/39), the editor correctly returned `LOCATION_NOT_FOUND`.

Diagnosis: rather than adding a line-mapping layer to paper over the mismatch, the actual source file was inspected. The real editable color literals were at different lines than the fixture claimed. This confirmed the editor pipeline itself was sound — the fixture was wrong, not the tool.

Fix: the fixture was corrected: `file` changed to `"src/main.tsx"`, and the two color locations updated to lines 77 and 83 respectively.

Verification: editor test suite (8 cases at this point) covering JSX string replacement, numeric expression replacement, boolean/null replacement, duplicate-value disambiguation by line, formatting preservation outside

the patched range, and each of the defined error conditions. All passing.

Step 7 — Editable panel and write-back

Built: the previously independent pieces connected end to end. `GenerationTrace`'s argument fields became real inputs, confirmed at `GenerationTrace.tsx:185`, each with its own input and Save action. The save flow, confirmed at `GenerationTrace.tsx:80`, reads the current argument, converts the input draft to the correct primitive type, calls `editSourceFile()`, and — only on success — calls `updateSourceRefArg()` to reflect the change locally.

The client layer (`packages/overlay/src/sourceEditClient.ts:51,68`) performs a read-then-write sequence: it fetches the current file content from `/__cts/read-file`, then posts `{file, line, argName, newValue, content}` to `/__cts/write-file`. The Vite endpoint calls the Step 6 `editSource()` directly (confirmed via source import at `vite-file-io.ts:4`) rather than duplicating any parsing logic, then writes the result with `fs.writeFile()`.

Problem found: none blocking at this step specifically — implementation proceeded cleanly once Step 6's fixture correction was in place. A React HMR root-recreation issue surfaced later, during Step 8 validation (see below).

Verification: live edits confirmed — `hotpink -> rebeccapurple` and `cyan -> gold` — with source-file inspection confirming only the intended literal changed in each case.

Step 8 — Edit -> HMR -> scene verification

Built: no new architecture; this step validated the full loop under realistic use: camera movement, rapid selection switching, untagged/background behavior, and repeated edits, all together.

Problem found (1): React HMR root recreation. Repeated HMR execution triggered repeated `createRoot()` calls and DOM-removal errors.

Diagnosis: the example entry point was inspected directly and found to recreate the React root on every HMR pass rather than reusing the existing one.

Fix: the entry point was changed to reuse the existing root across HMR updates. No custom scene-reload mechanism was introduced; the system continues to rely on Vite's own HMR pipeline, consistent with the Tech Stack doc's explicit design decision to reuse Vite's existing mechanism rather than build a parallel one.

Problem found (2): stale metadata after HMR — the substantial finding of this step, addressed in full detail in Section 5 below.

Verification: full manual pass — pink and cyan editing, camera orbit and zoom, rapid switching, untagged/background clearing, editing immediately after rapid selection changes, and selection/highlighting correctness after HMR. Automated checks (TypeScript builds, core/overlay tests, endpoint tests, Vite build, `git diff --check`) all passing.

Step 9 — Final success-criterion validation

Tested: the Stage 3 success criterion from the Proposal document — *"within 30 seconds of clicking an object, a developer who did not write the scene can find and edit the parameter responsible for its appearance."* Because an independent fresh developer was not available for this test, it was explicitly run as a **strict developer simulation** rather than an independent user study — a distinction worth preserving rather than overstating.

Result: pink cube parameter identified and edited in 2.424 seconds; cyan sphere in 2.279 seconds. Both substantially under the 30-second threshold. Source and rendered object both updated correctly in each case.

Initial verdict at this point: PASS WITH LIMITATION — criterion satisfied, but the stale-metadata behavior documented in Step 8 remained unresolved. That limitation was subsequently investigated and fixed (Section 5), so this verdict should be read historically: the criterion passed even under the limitation, and the limitation was later removed.

5. Stale Metadata After HMR — Full Investigation

This issue merits its own section because it was initially mistaken for a possible architectural limitation of the whole system, rather than what it actually was.

Observation: after an edit and HMR reload, the source file and the rendered object both correctly reflected the new value — but `GenerationTrace`, on reselection, could still show the original hardcoded value.

Two competing hypotheses were considered before any fix was attempted:

- **Hypothesis A — fixture/demo-data duplication.** The demo's `SourceRef.args.color` was a separately hardcoded value, independent of the actual JSX color prop. HMR re-executing the file would recreate that hardcoded value regardless of what the real prop had become.
- **Hypothesis B — a real resolver or state-management defect.** The resolver, the Zustand store, or the object-reference chain itself could have been retaining or manufacturing stale data — a materially more serious problem, since it would implicate the core engine rather than one demo file.

Diagnosis: the actual resolver, store, and `main.tsx` were inspected directly. No evidence was found that the resolver or Zustand were producing stale values on their own — the stale value was being freshly recreated by the fixture itself, every time. Root cause: **Hypothesis A**, fixture/demo-data duplication — not a resolver or state-management defect.

Fix: the fixture was restructured so the color value is no longer duplicated. `SourceRef.args` for both objects now starts empty; a fixture-only wrapper component (`SourceRefMaterial`, confirmed in source at `main.tsx:96`) derives `sourceRef.args.color` from the same `color` prop that actually drives the rendered material:

```
sourceRef.args = { ...sourceRef.args, color };  
return <meshStandardMaterial color={color} />;
```

The resolver, store, highlight component, Vite routes, and source editor were not touched — the fix was contained entirely to the demo fixture, correctly matching where the root cause was actually located.

Re-verification across two edit cycles per object, confirming the fix held rather than only working once:

Object	Edit 1	Edit 2	Result
Pink cube	hotpink -> rebeccapurple	rebeccapurple -> royalblue	Trace correctly showed the current value after reselection at each step
Cyan sphere	cyan -> gold	—	Trace correctly showed gold after reselection

The fixture was restored to its original hotpink/cyan values after testing.

A code-level discrepancy surfaced during the later audit, worth recording precisely: lines 77 and 83 are no longer raw `<meshStandardMaterial color="...">` elements as they were when Step 6's fixture correction was first made — they are now `<SourceRefMaterial ... color="...">` props, with the actual `<meshStandardMaterial>` rendered separately inside that wrapper (confirmed at [main.tsx:107](#)). This is a natural consequence of the Section 5 fix itself and does not affect correctness — the editor tests continue to pass against the current structure — but it is noted here so the exact shape at those line numbers is not assumed to have stayed static across this report's history.

6. What Was Actually Fixed vs. What Remains a Real, Documented Limitation

This distinction matters, and the two should not be collapsed into one.

6.1 Fixed and re-verified

The stale-metadata behavior described in Section 5 is resolved for the specific case it was tested against: **edits made through the panel's own Save action**. `handleSave()` explicitly calls `updateSourceRefArg()` on success, which is why the two-edit-cycle re-verification held.

6.2 A real, distinct limitation — not previously stated this precisely

A close reading of the reactive wiring ([GenerationTrace.tsx:52,58,98](#)) establishes something more specific than "HMR causes staleness," and it was not fully separated from the Section 5 fix until this consolidated report:

There is no mechanism that reconciles the Zustand store with actual file/object state after any change that does not go through the panel's own `updateSourceRefArg()` call. Specifically:

- The React-root-reuse fix (Section 4, Step 8) only re-renders the component tree on HMR. It does not call `resolveSourceRef()` or `select()` again.
- If the source file changes by any path other than the panel's Save button — a manual edit in an external editor, or, relevantly for Stage 4, a change originating from the real Procedural World Generator's own logic — the currently selected object's displayed `sourceRef` will not refresh automatically. Deselecting and reselecting the object is currently required to see the fresh value.

This was not exercised by the two-edit-cycle re-verification in Section 5, which only tested edits originating from the panel itself. It is accurately described as **unverified for the general case**, not merely "slow to update" —

there is currently no code path that would refresh it automatically at all, for any trigger other than the panel's own save.

6.3 Scope of what a `SourceRef` currently exposes

Separately from the reconciliation question: the panel only displays and edits whatever fields a given object's `SourceRef.args` was manually written to contain. For the current demo, that is `color` alone. The system does not discover "every value that affects a mesh" on its own — that remains explicitly the province of the not-yet-designed Stage 6.5 auto-instrumentation, and nothing in Stage 3 changes that boundary. Adding more trackable fields (e.g. scale, position) to a given generator today requires the same manual pattern already used for `color` — the editor pipeline itself is generic per-argument and already supports this without further engine changes.

7. Three.js Dependency Duplication — Full History

7.1 Discovery

The workspace initially declared different Three.js versions per package (`core: ^0.170.0`, `overlay` and `examples: ^0.185.1`), resulting in multiple physical installations and Three.js's own runtime warning about multiple imported instances.

7.2 A specific claim was checked and rejected, not assumed

One diagnosis suggested the duplication could cause `instanceof THREE.Object3D`-style identity check failures. This was not accepted on faith. The actual Click-to-Source source was searched directly for `instanceof THREE.Object3D`, `instanceof THREE.Mesh`, `THREE.isObject3D`, and any other constructor-identity check. None exist anywhere in the codebase — the resolver works structurally, via `userData` and `parent` traversal, and `OutlinePass` operates on object references rather than identity checks. **The specific `instanceof` failure claim was therefore ruled out for this project.**

7.3 What remained a genuine problem regardless

Even without the specific `instanceof` failure, multiple physical Three.js copies in one dependency tree is a legitimate correctness risk in general (objects crossing runtime boundaries can silently fail structural checks in other, less obvious ways) and was corrected on that basis.

7.4 Normalization

The workspace was normalized to `three@0.170.0` — the version already declared by `core`, making it the version requiring the least change to converge on. This was a build-topology decision, not a claim that `0.170.0` was inherently the better or newer choice.

7.5 Peer-range verification — resolved

A later, code-level check confirmed this decision was also compatible, not merely convenient. Read directly from `node_modules/@react-three/fiber/package.json`: version `9.7.0`, declared peer dependency

"three": ">=0.156". Installed Three.js: 0.170.0. Overlay's own declared peer range: >=0.170.0. three@0.170.0 satisfies both ranges — confirmed by direct package-metadata inspection, not inferred. npm ls @react-three/fiber --all separately failed with an unrelated internal npm error during this check; that failure does not affect the conclusion above, since the peer range was read directly from the package's own package.json rather than depended on that command's output.

7.6 Verification

Post-reinstall, npm ls three showed a single deduplicated three@0.170.0; npm ls @types/three showed a single @types/three@0.170.0. The multiple-instance warning no longer appears. All tests and builds passed after normalization.

8. Final Verification State

Check	Result
Core tests (resolver + source editor)	14 passed
Overlay tests (store + edit client)	7 passed
File-I/O / endpoint tests	4 passed
Shared / core / overlay TypeScript builds	Passed
Root npx tsc -b	Passed (references shared, core, overlay; examples remains Vite-built, not part of the root TS solution)
Examples Vite build	Passed
git diff --check	Passed (pre-existing line-ending warnings only)
Three.js runtime duplication	Resolved and verified
React HMR root recreation	Resolved and verified
Stale metadata (panel-originated edits)	Resolved and re-verified across two edit cycles per object
Stale metadata (non-panel-originated changes)	Not resolved — no reconciliation path exists; documented as an open limitation
Success criterion (developer simulation)	Passed — 2.424s and 2.279s, both well under the 30s threshold

A minor housekeeping note surfaced during the code-level audit: running each package's default npm test script currently double-counts generated JavaScript copies of the same test files alongside the canonical TypeScript sources (reporting 28 and 14 respectively, versus the canonical 14 and 7). This does not indicate additional untested code — the canonical, explicitly-targeted TypeScript test run is the accurate count of 14 core + 7 overlay + 4 file-I/O = 25 tests, all passing. Worth cleaning the test glob before Stage 4 so future runs report the true count without requiring this caveat.

9. Documentation History Note

An earlier version of this report declared Stage 3 complete after Steps 1 through 5 only — selection, highlighting, and read-only trace display. That declaration was incorrect: editing and write-back are stated as part of the Stage 3 MVP in the Final Approach, Proposal, and Tech Stack documents alike, not deferred the way automatic instrumentation was. That earlier declaration was corrected before being treated as final, and this report reflects the complete ten-step implementation rather than the earlier partial one. This correction is recorded here deliberately, consistent with how Stage 1's methodology error (Experiment 5) and Stage 2's audit scope were both handled — as a documented correction, not a silently revised history.

10. Final Assessment

Stage 3 converted the Stage 2 architectural contract into a working system: a developer can click a rendered object, see exactly the source file, function, line, and manually-declared arguments responsible for its current appearance, edit one of those values, and watch the change land on disk and reappear in the running scene through ordinary Vite HMR — with the loop measured at roughly 2.3–2.4 seconds end to end against the project's own 30-second success criterion.

Two matters are carried forward as explicit, honestly-scoped items rather than either hidden or overstated. First, the system has no path to refresh a selected object's displayed metadata when the underlying source changes by any means other than the panel's own edit action — worth real attention once Stage 4 introduces a generator (the real Procedural World Generator) whose own logic, not just manual panel edits, may change values during dogfooding. Second, what a given object exposes as trackable and editable remains entirely up to manual declaration, by design — Stage 3 proves the mechanism works for whatever is declared, and does not attempt to discover provenance on its own. That capability remains correctly deferred, undesigned, and unscheduled with certainty at Stage 6.5.

Stage 3 is functionally complete against its own ten-step definition and ready to support Stage 4.